

Operating Systems - Chapter 17 Summary

Free-Space Management

- Free-space management is fundamental to allocators (malloc/free) and OS memory systems.
- Easy with fixed-size units (paging) → just keep a list of free pages.
- Hard with variable-size units (malloc, segmentation) → leads to external fragmentation.

Assumptions

- Focus on malloc/free interface.
- Concerned mostly with external fragmentation (unused holes between allocations).
- Allocated memory cannot be relocated → no compaction at user-level.
- Allocator manages contiguous region (the heap).

Low-level Mechanisms

- Splitting: divide a larger free block into allocated part + remainder.
- Coalescing: merge adjacent free blocks on free() to reduce fragmentation.
- Headers: each allocated block has a header storing size and metadata (e.g., magic number).
- Free lists: track free chunks by embedding list nodes inside free space.

Growing the Heap

- If heap runs out, allocator may request more memory from OS (e.g., sbrk, mmap).

Basic Allocation Strategies

- Best Fit: choose smallest block $\geq$ request. Reduces wasted space but may create tiny fragments; slow search.
- Worst Fit: choose largest block. Leaves larger chunks available, but performs poorly in practice.
- First Fit: choose first block that fits. Fast but can fragment the start of the heap.
- Next Fit: like first fit but continue from last position; spreads allocations more evenly.

Other Approaches

- Segregated lists: maintain separate lists for popular sizes; reduces fragmentation, speeds up allocation.
- Slab allocator (used in Solaris): caches kernel objects, keeps them pre-initialized for speed.
- Buddy allocator: splits memory into power-of-two blocks; easy coalescing via 'buddy' rule.
- Advanced allocators: use trees or more complex structures for scalability on multiprocessors.

Summary

Free-space management deals with satisfying variable-sized allocation requests while minimizing fragmentation. Mechanisms include splitting, coalescing, and free lists. Strategies like best-fit, worst-fit, first-fit, and next-fit have trade-offs in speed and fragmentation. More advanced methods include

segregated lists, slab allocation, and buddy systems. Building scalable allocators remains a challenge in modern systems.